Recitation 5

Reichman University
Efi Arazi School of Computer Science

Recitation overview

- Basic input and output ←
- Compilation – dealing with errors and warnings

Input and output

- C programs have two forms of output:
  - Exit status (integer return value of main)
  - Output printed to file / output stream
- C programs have two forms of input:
  - Command line arguments (list of strings)
  - Input read from file / `stdin`
- Today we will see output / input using `stdout` / `stdin`
- The function `printf()` prints to `stdout`
- The function `scanf()` reads from `stdin`
- Include `stdio.h` to use either of them

Output - the `printf` function

- The first argument is the control string
- The control string contains text and **conversion specifiers** (indicated by %) for injecting variable content
- For each conversion specifier in the control string, there is an additional argument (in order)

```
int i = 5;
printf ("Arithmetic operation:\n");
printf ("%d + %d = ", 3, i);
printf ("%d\n", 3+i);
```

← explicitly specify newline

← any expression can be used

- `printf` has a variable number of parameters (uncommon in C)
- `printf` returns the length of the string it printed (including newline)

Conversion specifiers

Specifier	Corresponding argument
`%c`	Character
`%d`	Decimal integer
`%e`	Floating point – scientific notation
`%f`	Floating point
`%lf`	Long float (double)
`%s`	String

Defining field width

- With `printf()`, it is possible to define the field width for each of the printed arguments
- This is done by adding a number before the conversion specification. The value is aligned to the right and padded on the left with spaces.

```
printf ("Chars:%3c%3c%3c\n", 'a', 'b', 'c') ;
printf ("Twelve:%6d\n", 12);
printf ("Two thirds:%6.3f\n", (2.0 / 3));
```

```
Chars:  a  b  c
Twelve:    12
Two thirds: 0.667
```

Defining field width

- Default is that the value is aligned to the right and padded to the left with spaces.
  - use '–' before the field width for left aligned printing
  - use '0' to pad the left with zeros

```
int a = 5, b = 12;

printf ("%d,%d\n", a, b);
printf ("%4d,%4d\n", a, b);
printf ("%-4d,%-4d\n", a, b);
printf ("%04d,%04d\n", a, b);
```

```
5,12
   5,  12
5   ,12
0005,0012
```

Input - the `scanf` function

- The function **`scanf()`** reads from **`stdin`** and, like **`printf()`**, its first argument is a control string with conversion specifiers

```
scanf ("%d", &x);
```

- The symbol '**`&`**' is the address operator – the value read is written in the memory space of variable **`x`**.
- The '**`%d`**' tells **`scanf()`** to try to read an integer from the standard input

- **`scanf`** has a variable number of parameters
- **`scanf`** returns the number of values successfully read as input (should be number of arguments -1)

Input and output – example 1

```
#include <stdio.h>

int main() {

   int n1, n2;
   printf ("Input two integer numbers:");
   scanf ("%d%d", &n1, &n2);
   printf ("\nThe sum of the two numbers is:\n");
   printf ("%d+%d=%d\n", n1,n2,n1+n2);
   return 0;
}
```

← Variables should be defined at top of function (unenforced convention)

code in `/share/recitations/recitation05/sum.c`

Input and output – example 1

```
#include <stdio.h>

int main() {

   int n1, n2, res;

   printf ("Input two integer numbers:");
   res=scanf ("%d%d", &n1, &n2);
   if(res<2) {
      printf("Wrong input. Expecting two integers.\n”);
      return 1;   ← Exit status ≠0 marks error
   }
   printf ("\nThe sum of the two numbers is:\n");
   printf ("%d+%d=%d\n", n1,n2,n1+n2);
   return 0;
}
```

code in `/share/recitations/recitation05/sum-with-check.c`

Input and output – example 2

```
#include <stdio.h>

int main() {

   char     c1, c2;

   printf ("Input two characters:");
   scanf ("%c%c", &c1, &c2);
   printf ("\nThe data that you typed in:\n");
   printf ("%3c%3c\n", c1, c2);
   return 0;
}
```

- For characters – `scanf()` reads in sequence
- Other types – `scanf()` reads with white space separators
- Better to use file parsing functions, which we see later in the course.

Recitation overview

- Basic input and output
- Compilation – dealing with errors and warnings←

Your first C program

```
#include <stdio.h>

int main() {
   printf ("Hello, world!\n");
   return 0;
}
```

code in `/share/recitations/recitation05/hello.c`

- Compile this source file using **`gcc`**:

```
==> gcc -Wall -o hello hello.c
```

By convention, the executable typically has the same name as the C file, but without the '`.c`' extension

- Compilation produces an executable (**`hello`**) that we can run:

```
==> ./hello
```

```
Hello, world!
```

Dealing with compilation errors and warnings – cprog.c

- After we fixed the error and warnings
  - Add new line printing in the last printf for better printing
  - See what happens when we pipe 2 numbers in to the runnable program
  - See what happens when we insert floating point numbers
  - See what happens when we insert non-numeric values
    - Now you've realized that we need to check our input, so we'll use (scanf("%d %d",&i,&j)==2) to check if the scan was successful